Atelier 3

Gestion d'un centre de formation avec procédures et fonctions stockées

Base de données MySQL (Aiven) : techlearn_db

Rapport de travaux pratiques

14 novembre 2026

1. Contexte et objectifs pédagogiques

Contexte

Le centre de formation TechLearn propose plusieurs sessions animées par des formateurs. Chaque session a un formateur, une capacité maximale et un nombre d'inscrits. L'administration veut automatiser quelques tâches : calculer le taux d'occupation d'une session, mettre à jour le nombre d'inscrits, supprimer les sessions vides et générer des rapports.

Pour cela, j'ai créé une base de données MySQL nommée **techlearn_db**, hébergée sur Aiven, avec trois tables : **formateurs**, **stagiaires** et **sessions**. J'ai fait tout le travail depuis le terminal, avec le client mysql.

Objectifs pédagogiques

- Créer des procédures et des fonctions stockées.
- Utiliser les paramètres IN, OUT et INOUT.
- Utiliser des structures de contrôle : la boucle WHILE et la condition IF.
- Automatiser des tâches directement dans MySQL.
- Documenter le travail dans un rapport technique avec le code et des captures d'écran.

Organisation du rapport

Dans la partie 2, chaque étape est présentée de la même façon : ce que je fais, le code SQL, la capture d'écran de l'exécution et l'analyse du résultat. Ainsi, le code complet de l'atelier se retrouve dans l'ordre où je l'ai exécuté.

2. Réalisation, étape par étape

2.1 Connexion à Aiven

Je me connecte au serveur MySQL d'Aiven depuis le terminal avec le client mysql. J'utilise le certificat ca.pem pour la connexion sécurisée (SSL). Le mot de passe est demandé juste après la commande. Ensuite, j'affiche les bases existantes.

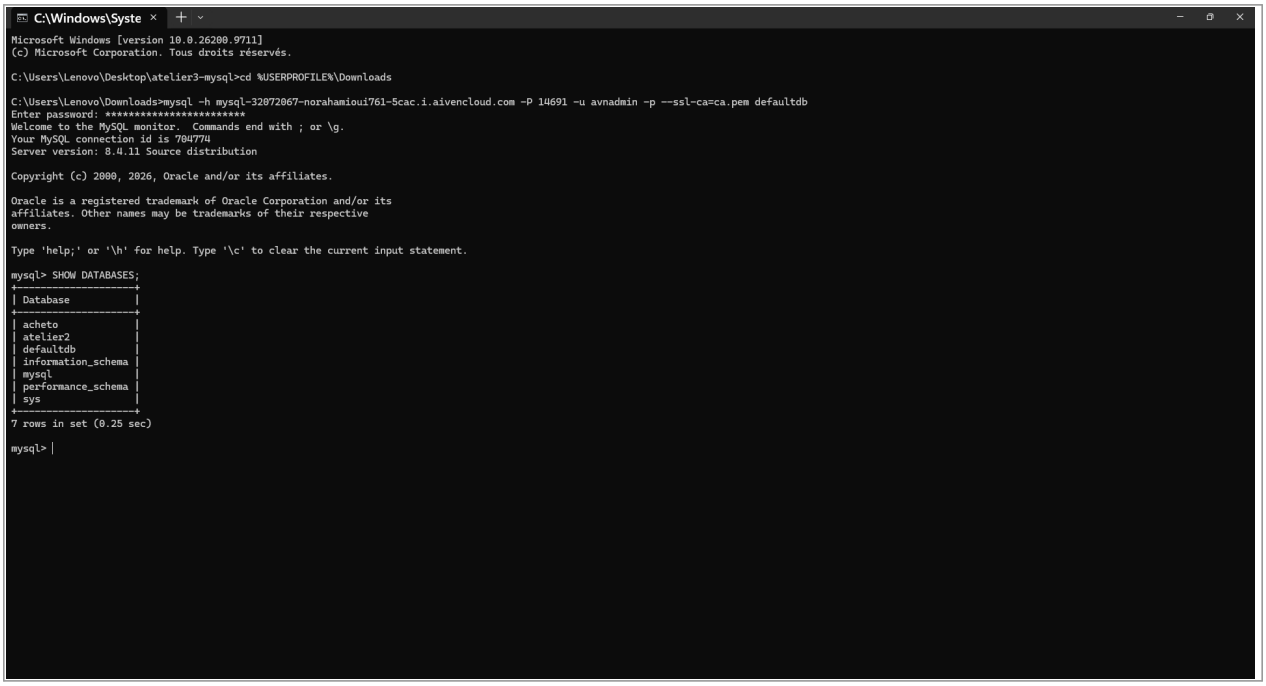
Code

```
mysql -h mysql-32072067-norahamioui761-5cac.i.aivencloud.com -P 14691 -u avnadmin -p --ssl-ca=ca.pem defaultdb

SHOW DATABASES;
```

Capture d'écran

Capture 1 : connexion à Aiven et liste des bases



Résultat

La connexion fonctionne (serveur MySQL 8.4.11) et SHOW DATABASES affiche 7 bases. La base techlearn_db n'existe pas encore.

2.2 Création de la base techlearn_db

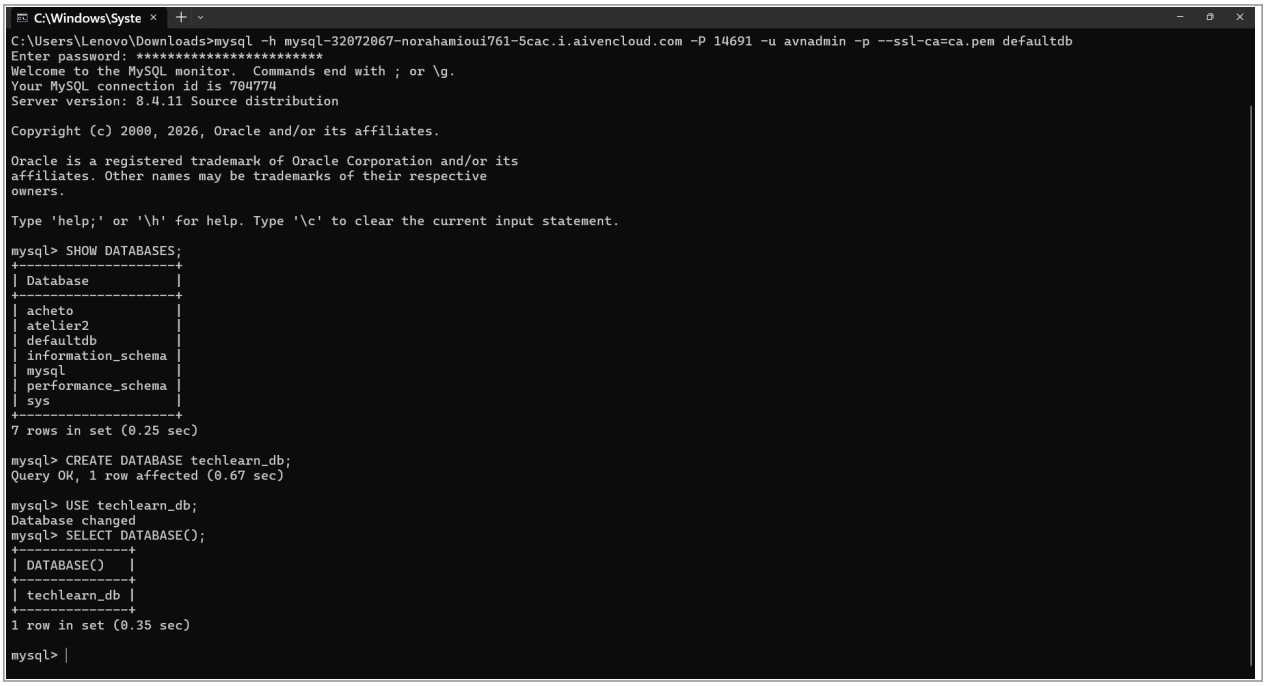
Je crée la base de données de l'atelier, puis je la sélectionne avec USE. La commande SELECT DATABASE() permet de vérifier que je travaille bien dans techlearn_db.

Code

```
CREATE DATABASE techlearn_db;
USE techlearn_db;
SELECT DATABASE();
```

Capture d'écran

Capture 2 : création de la base et vérification



Résultat

Le terminal affiche Query OK pour la création, puis Database changed. SELECT DATABASE() retourne techlearn_db.

2.3 Création des tables

Je crée les trois tables. La table sessions contient une clé étrangère : formateur_id fait référence à l'id de la table formateurs. Chaque session est donc liée à un formateur. Je vérifie avec SHOW TABLES.

Code

```
CREATE TABLE formateurs (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  specialite VARCHAR(100)
);

CREATE TABLE stagiaires (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nom VARCHAR(100),
  ville VARCHAR(100)
);

CREATE TABLE sessions (
  id INT AUTO_INCREMENT PRIMARY KEY,
  intitule VARCHAR(100),
  formateur_id INT,
  capacite INT,
  inscrits INT DEFAULT 0,
  FOREIGN KEY (formateur_id) REFERENCES formateurs(id)
);

SHOW TABLES;
```

Capture d'écran

Capture 3 : création des trois tables



Résultat

Les trois CREATE TABLE donnent Query OK. SHOW TABLES affiche formateurs, sessions et stagiaires.

2.4 Insertion des données

J'insère 3 formateurs, 3 stagiaires et 3 sessions. Ensuite, j'affiche chaque table avec SELECT pour vérifier le contenu.

Code

```
INSERT INTO formateurs (nom, specialite) VALUES
('Mme Nadia', 'Bases de données'),
('M. Karim', 'Développement Web'),
('Mme Salma', 'Analyse de données');

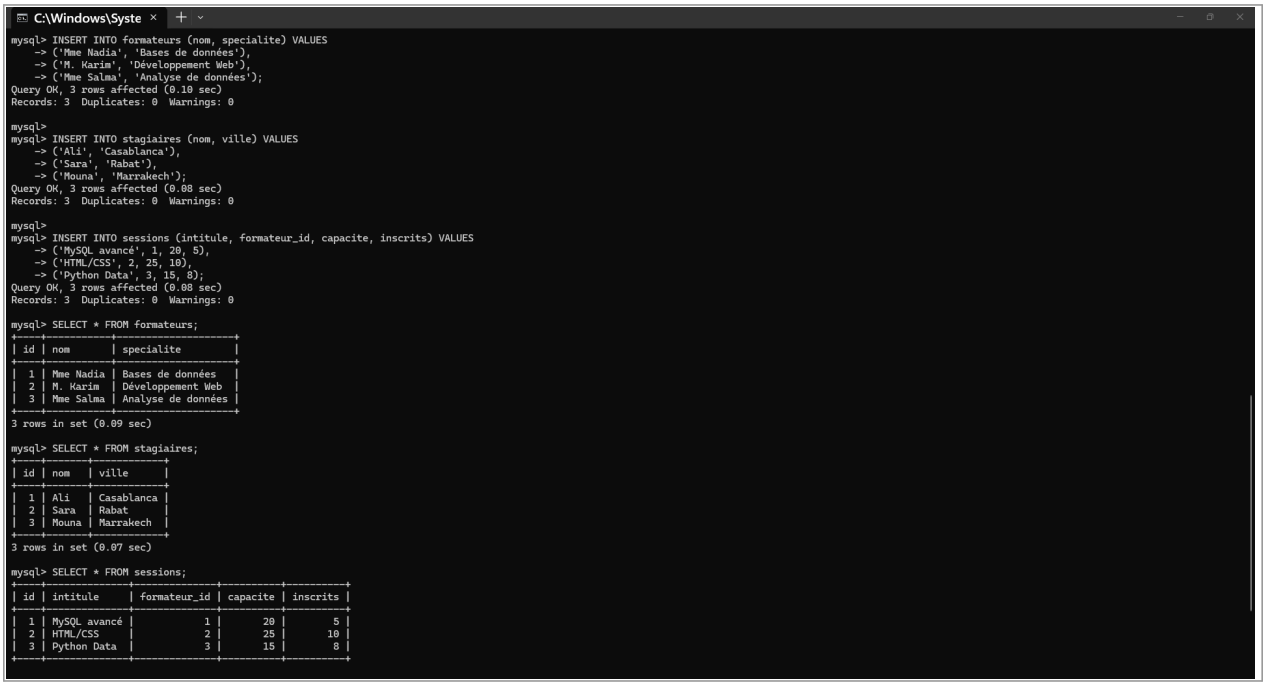
INSERT INTO stagiaires (nom, ville) VALUES
('Ali', 'Casablanca'),
('Sara', 'Rabat'),
('Mouna', 'Marrakech');

INSERT INTO sessions (intitule, formateur_id, capacite, inscrits) VALUES
('MySQL avancé', 1, 20, 5),
('HTML/CSS', 2, 25, 10),
('Python Data', 3, 15, 8);

SELECT * FROM formateurs;
SELECT * FROM stagiaires;
SELECT * FROM sessions;
```

Capture d'écran

Capture 4 : insertion et affichage des données



Résultat

Chaque INSERT affiche 3 rows affected, et chaque table contient bien 3 lignes. Au départ, les sessions ont 5, 10 et 8 inscrits.

2.5 Exercice 1 : procédure avec paramètre IN

La procédure afficher_stagiaires_par_ville reçoit une ville en paramètre IN et affiche les stagiaires de cette ville. J'utilise DELIMITER // pour que MySQL comprenne que le corps de la procédure se termine à END //.

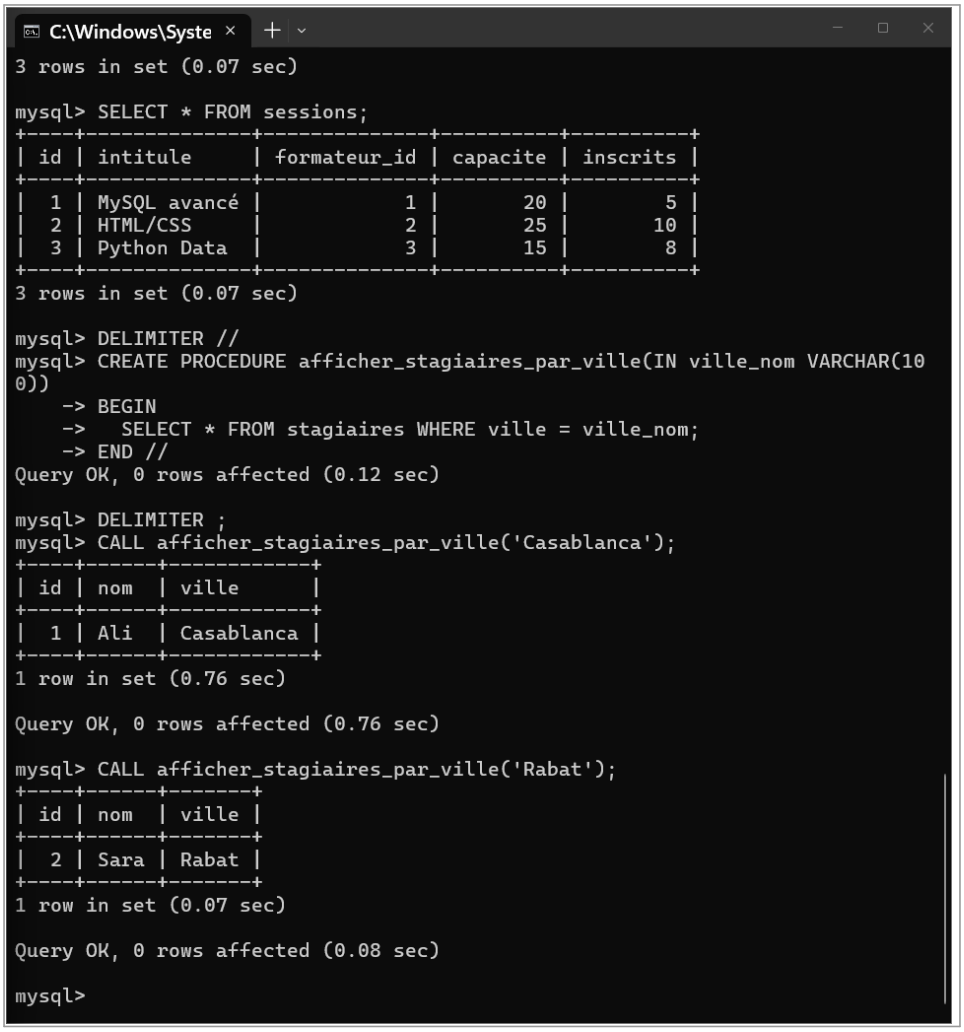
Code

```
DELIMITER //
CREATE PROCEDURE afficher_stagiaires_par_ville(IN ville_nom VARCHAR(100))
BEGIN
  SELECT * FROM stagiaires WHERE ville = ville_nom;
END //
DELIMITER ;

CALL afficher_stagiaires_par_ville('Casablanca');
CALL afficher_stagiaires_par_ville('Rabat');
```

Capture d'écran

Capture 5 : procédure avec paramètre IN



Résultat

Pour 'Casablanca', la procédure retourne Ali. Pour 'Rabat', elle retourne Sara. Le paramètre IN permet de choisir la ville à chaque appel.

2.6 Exercice 2 : procédure avec paramètre OUT

La procédure compter_sessions calcule le nombre de sessions et le met dans un paramètre OUT. On récupère ensuite la valeur dans une variable utilisateur (@nb).

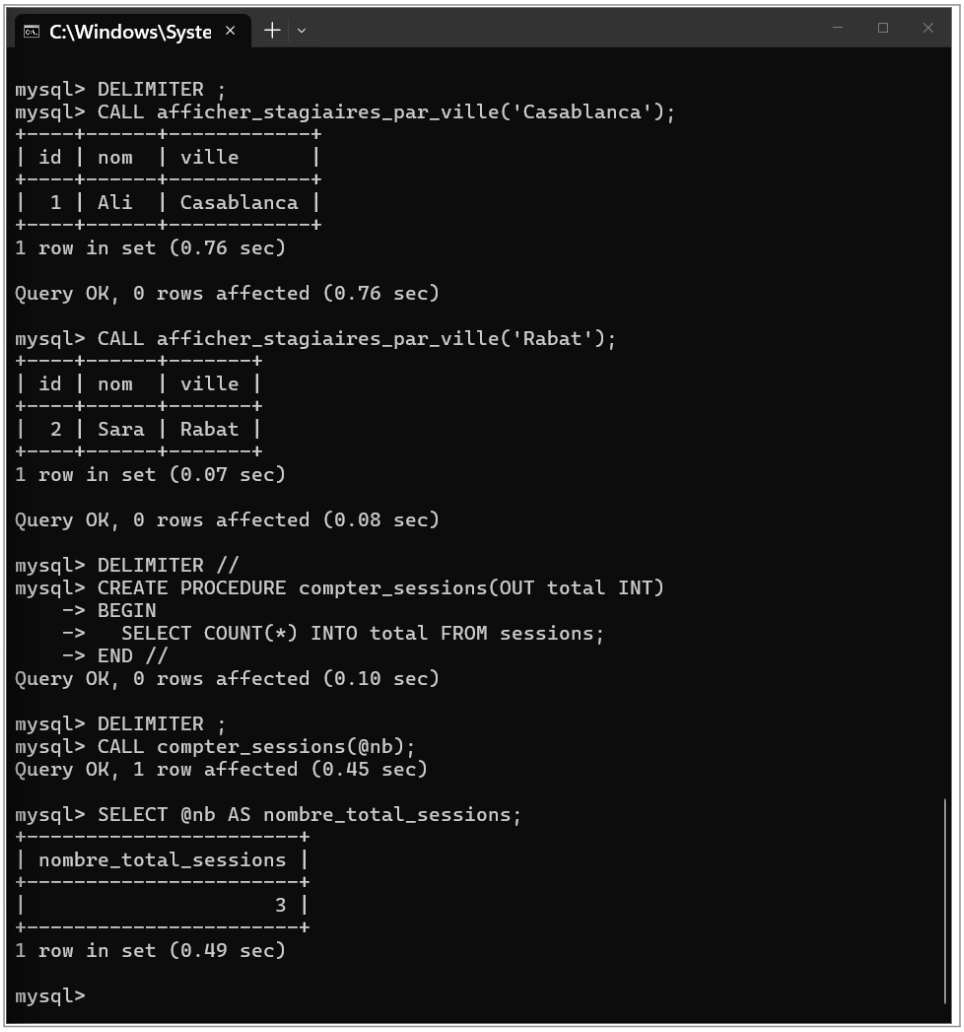
Code

```
DELIMITER //
CREATE PROCEDURE compter_sessions(OUT total INT)
BEGIN
  SELECT COUNT(*) INTO total FROM sessions;
END //
DELIMITER ;

CALL compter_sessions(@nb);
SELECT @nb AS nombre_total_sessions;
```

Capture d'écran

Capture 6 : procédure avec paramètre OUT



Résultat

Le résultat est 3, ce qui correspond aux 3 sessions insérées au départ. Le paramètre OUT sert à renvoyer un résultat à celui qui appelle la procédure.

2.7 Exercice 3 : procédure avec paramètre INOUT

La procédure inscrire_stagiaire ajoute 1 au nombre d'inscrits d'une session, puis renvoie le nouveau total dans le même paramètre. Le paramètre INOUT entre avec une valeur (0 au début) et ressort modifié.

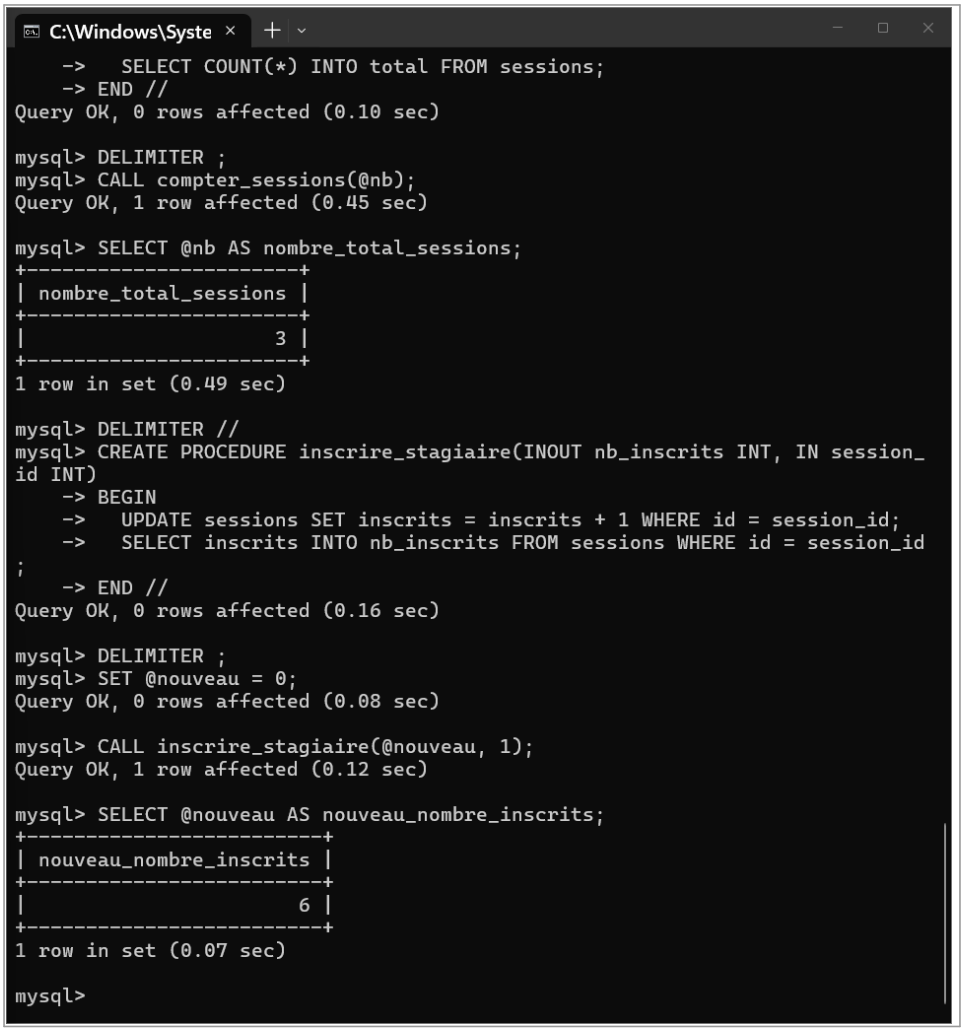
Code

```
DELIMITER //
CREATE PROCEDURE inscrire_stagiaire(INOUT nb_inscrits INT, IN session_id INT)
BEGIN
  UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
  SELECT inscrits INTO nb_inscrits FROM sessions WHERE id = session_id;
END //
DELIMITER ;

SET @nouveau = 0;
CALL inscrire_stagiaire(@nouveau, 1);
SELECT @nouveau AS nouveau_nombre_inscrits;
```

Capture d'écran

Capture 7 : procédure avec paramètre INOUT



Résultat

Pour la session 1, le nombre d'inscrits passe de 5 à 6. La variable @nouveau contient la nouvelle valeur.

2.8 Exercice 4 : fonction taux_occupation

Cette fonction calcule le taux d'occupation d'une session avec la formule (inscrits / capacité) x 100. Contrairement à une procédure, une fonction retourne une valeur et peut s'utiliser directement dans un SELECT. J'ai ajouté READS SQL DATA, car le serveur Aiven l'exige pour créer une fonction.

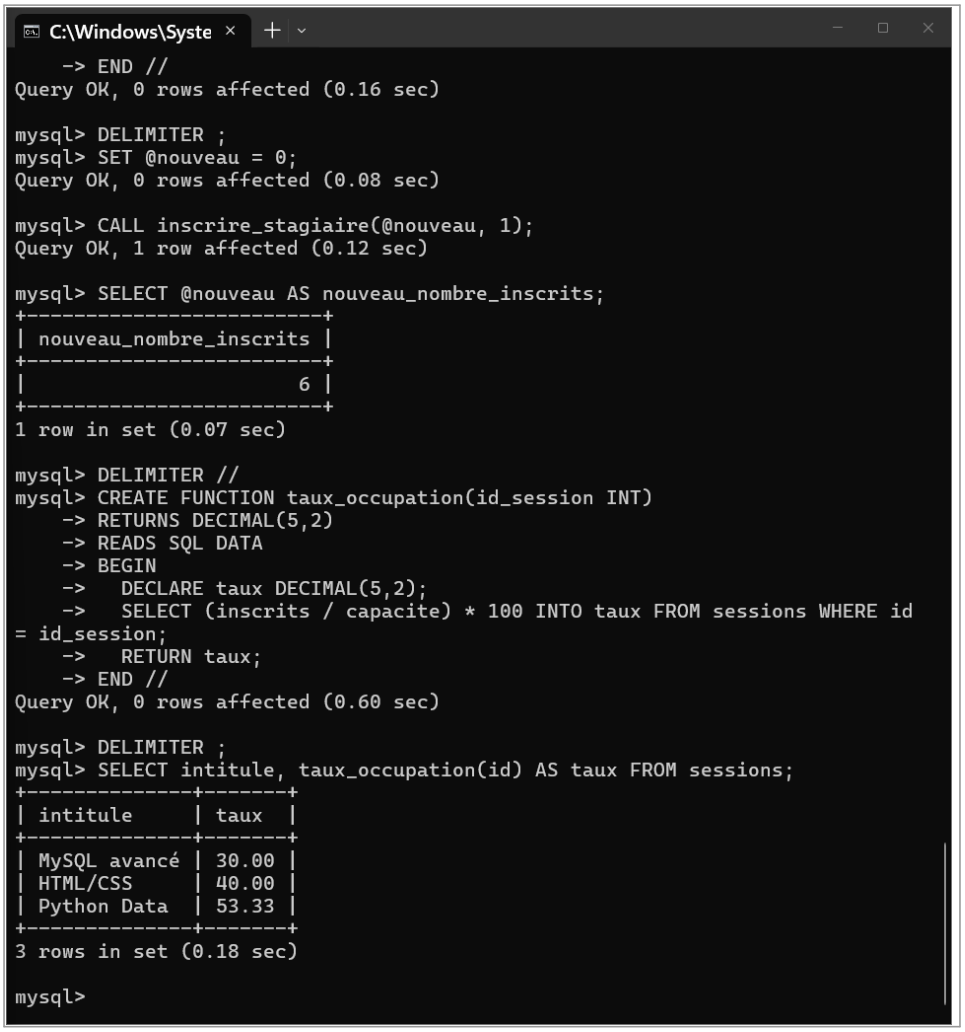
Code

```
DELIMITER //
CREATE FUNCTION taux_occupation(id_session INT)
RETURNS DECIMAL(5,2)
READS SQL DATA
BEGIN
  DECLARE taux DECIMAL(5,2);
  SELECT (inscrits / capacite) * 100 INTO taux FROM sessions WHERE id = id_session;
  RETURN taux;
END //
DELIMITER ;

SELECT intitule, taux_occupation(id) AS taux FROM sessions;
```

Capture d'écran

Capture 8 : fonction taux_occupation



Résultat

Les résultats sont 30.00 % pour MySQL avancé (6 sur 20), 40.00 % pour HTML/CSS (10 sur 25) et 53.33 % pour Python Data (8 sur 15).

2.9 Exercice 5 : boucle WHILE

La procédure generer_sessions utilise une boucle WHILE pour insérer 10 sessions d'un coup pour un formateur donné. La variable i compte de 1 à 10, et CONCAT sert à construire le nom de chaque session.

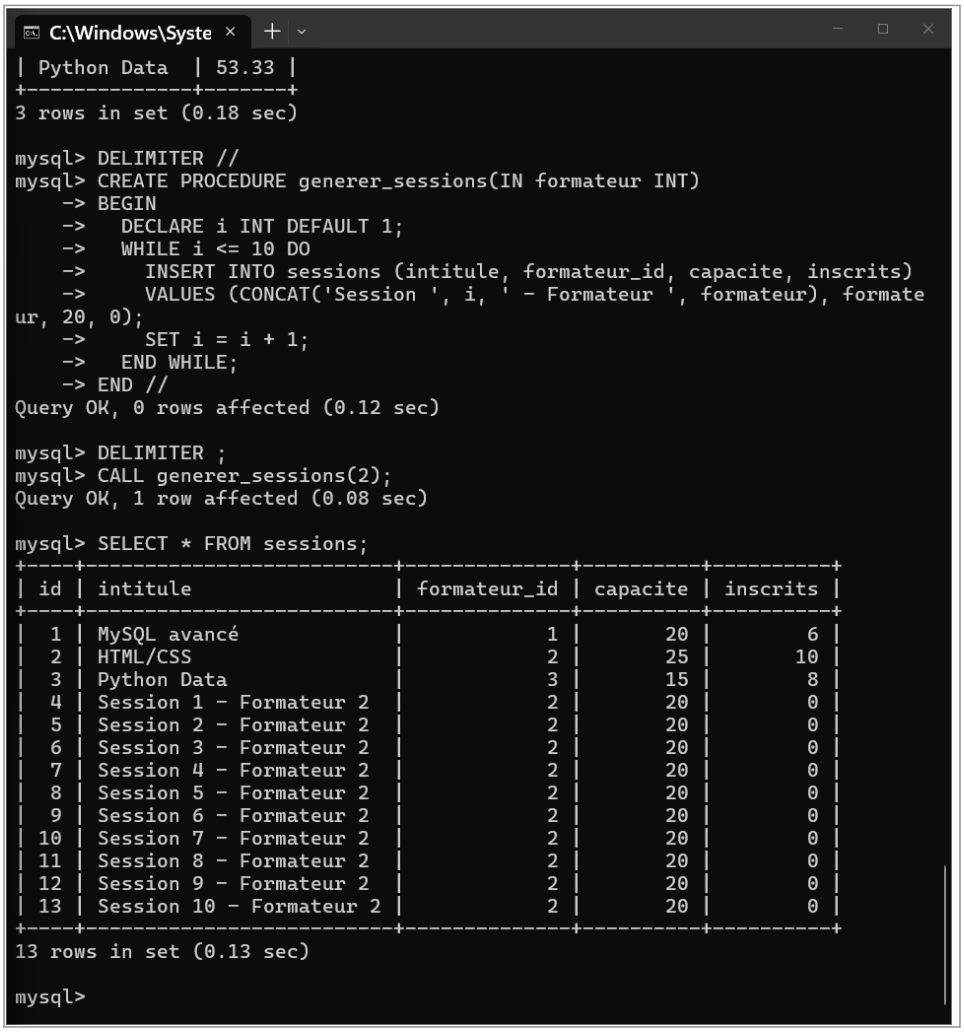
Code

```
DELIMITER //
CREATE PROCEDURE generer_sessions(IN formateur INT)
BEGIN
  DECLARE i INT DEFAULT 1;
  WHILE i <= 10 DO
    INSERT INTO sessions (intitule, formateur_id, capacite, inscrits)
    VALUES (CONCAT('Session ', i, ' - Formateur ', formateur), formateur, 20, 0);
    SET i = i + 1;
  END WHILE;
END //
DELIMITER ;

CALL generer_sessions(2);
SELECT * FROM sessions;
```

Capture d'écran

Capture 9 : boucle WHILE



Résultat

La table sessions passe de 3 à 13 lignes. Les 10 nouvelles sessions (Session 1 à Session 10 - Formateur 2) ont une capacité de 20 et 0 inscrit.

2.10 Défi 1 : fonction sessions_formateur

Cette fonction retourne le nombre de sessions d'un formateur à partir de son nom. Le nom est dans la table formateurs et formateur_id est dans sessions, donc j'utilise une jointure (JOIN).

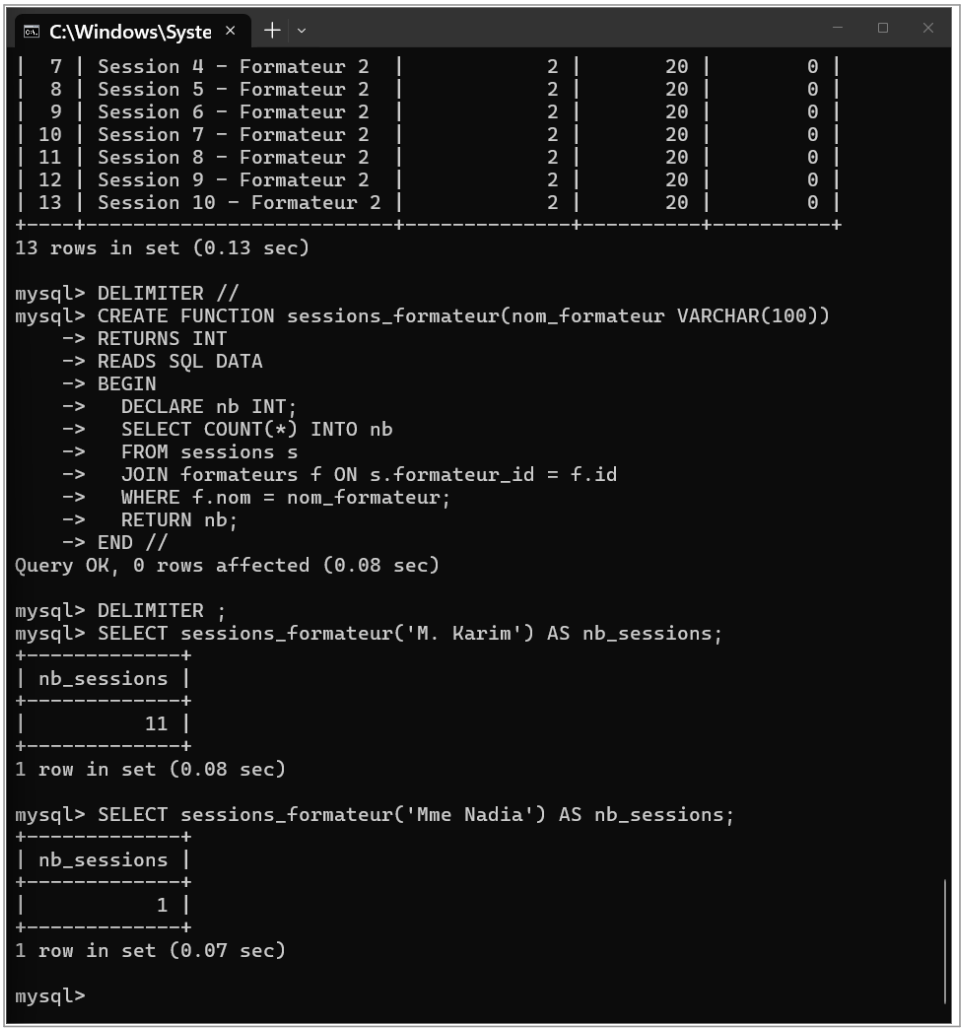
Code

```
DELIMITER //
CREATE FUNCTION sessions_formateur(nom_formateur VARCHAR(100))
RETURNS INT
READS SQL DATA
BEGIN
  DECLARE nb INT;
  SELECT COUNT(*) INTO nb
  FROM sessions s
  JOIN formateurs f ON s.formateur_id = f.id
  WHERE f.nom = nom_formateur;
  RETURN nb;
END //
DELIMITER ;

SELECT sessions_formateur('M. Karim') AS nb_sessions;
SELECT sessions_formateur('Mme Nadia') AS nb_sessions;
```

Capture d'écran

Capture 10 : fonction sessions_formateur



Résultat

La fonction retourne 11 pour M. Karim (1 session d'origine + 10 générées) et 1 pour Mme Nadia.

2.11 Défi 2 : procédure supprimer_sessions_vides

Cette procédure supprime toutes les sessions qui n'ont aucun inscrit (inscrits = 0).

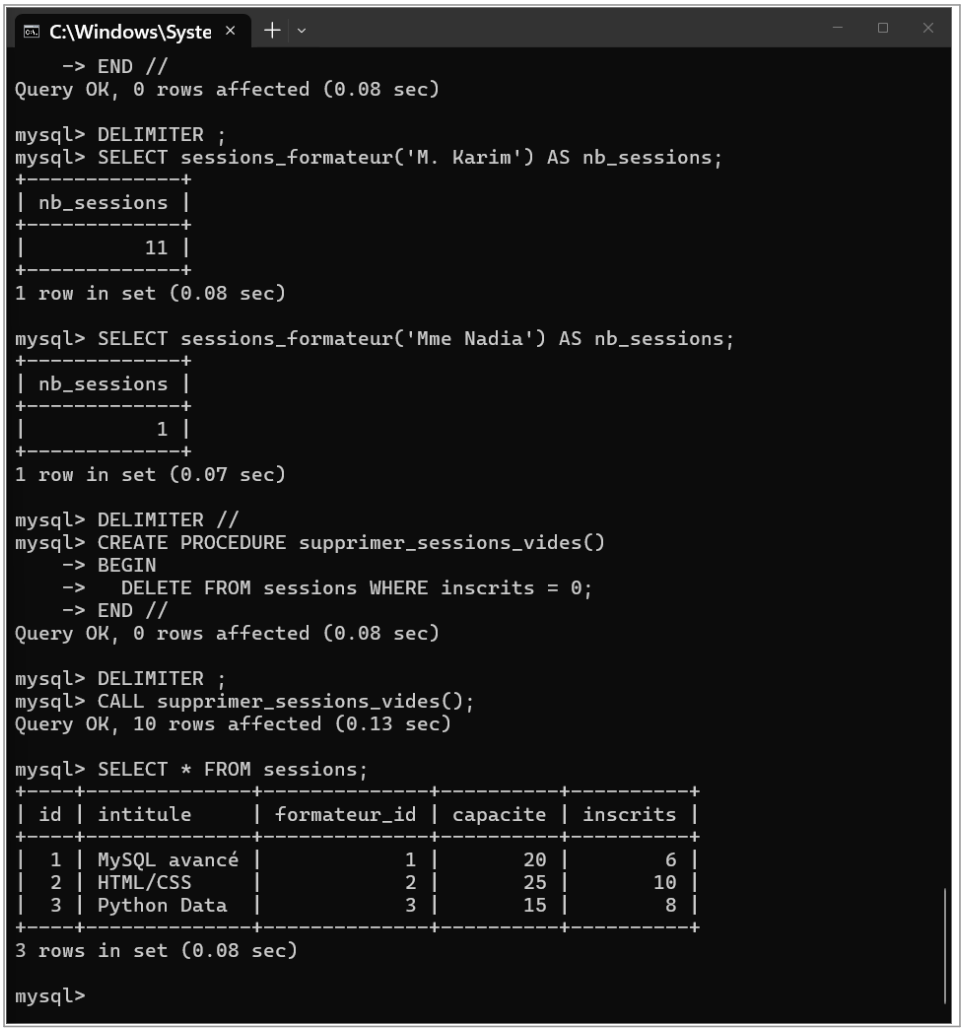
Code

```
DELIMITER //
CREATE PROCEDURE supprimer_sessions_vides()
BEGIN
  DELETE FROM sessions WHERE inscrits = 0;
END //
DELIMITER ;

CALL supprimer_sessions_vides();
SELECT * FROM sessions;
```

Capture d'écran

Capture 11 : suppression des sessions vides



Résultat

Le terminal indique 10 rows affected : ce sont les 10 sessions générées à l'exercice 5. Il reste les 3 sessions d'origine.

2.12 Défi 3 : procédure valider_inscription

Cette procédure utilise une structure IF. Si le nombre d'inscrits est supérieur ou égal à la capacité, elle affiche "Session complète". Sinon, elle ajoute 1 aux inscrits et affiche "Inscription validée". Pour tester les deux cas, j'ai mis la session 3 à 15 inscrits sur 15 avec un UPDATE.

Code

```
DELIMITER //
CREATE PROCEDURE valider_inscription(IN session_id INT)
BEGIN
  DECLARE nb INT;
  DECLARE cap INT;
  SELECT inscrits, capacite INTO nb, cap FROM sessions WHERE id = session_id;
  IF nb >= cap THEN
    SELECT 'Session complète' AS message;
  ELSE
    UPDATE sessions SET inscrits = inscrits + 1 WHERE id = session_id;
    SELECT 'Inscription validée' AS message;
  END IF;
END //
DELIMITER ;

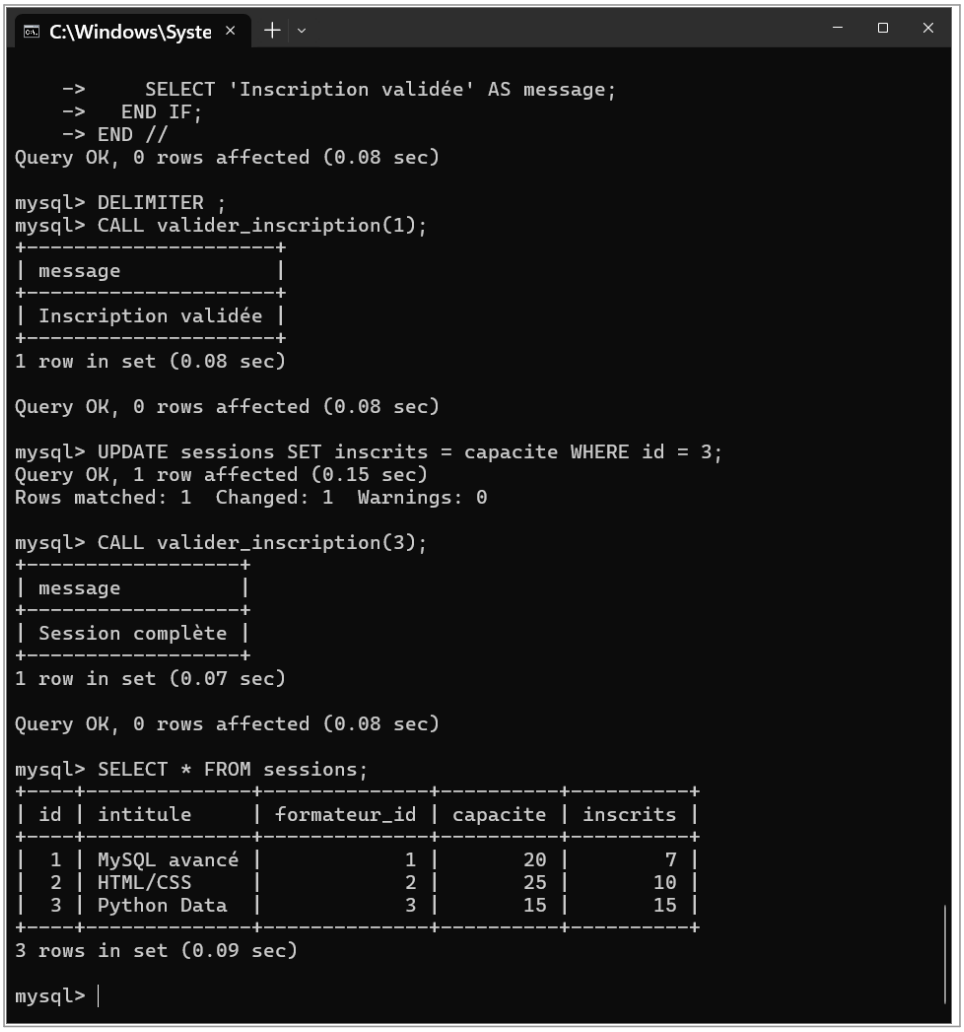
-- Cas 1 : session non complète
CALL valider_inscription(1);

-- Cas 2 : on remplit la session 3 pour tester "Session complète"
UPDATE sessions SET inscrits = capacite WHERE id = 3;
CALL valider_inscription(3);

SELECT * FROM sessions;
```

Capture d'écran

Capture 12 : procédure valider_inscription (deux cas)



Résultat

Cas 1 : la session 1 affiche "Inscription validée" et passe de 6 à 7 inscrits. Cas 2 : la session 3 (15 sur 15) affiche "Session complète" et ne change pas.

2.13 Remarques techniques

- J'ai utilisé **DELIMITER //** à la place de DELIMITER / pour éviter des problèmes dans le terminal.
- J'ai ajouté **READS SQL DATA** aux fonctions, car le serveur Aiven l'exige quand le journal binaire est activé (sinon on risque l'erreur 1418).
- Pour tester "Session complète", j'ai mis inscrits = capacite sur la session 3 avec un UPDATE.

3. Conclusion personnelle

Cet atelier m'a permis de bien voir la différence entre une procédure et une fonction stockée. Une procédure fait une action (afficher, modifier, supprimer), alors qu'une fonction retourne une valeur qu'on peut utiliser directement dans un SELECT, comme taux_occupation.

J'ai aussi compris les trois types de paramètres : IN pour envoyer une valeur à la procédure, OUT pour récupérer un résultat, et INOUT pour envoyer une valeur et la récupérer modifiée.

J'ai utilisé une boucle WHILE et une condition IF directement dans MySQL. Cela permet d'automatiser des tâches, comme créer 10 sessions d'un seul coup. J'ai aussi vu qu'il faut faire attention avec DELETE : la procédure supprimer_sessions_vides a supprimé toutes les sessions avec 0 inscrit.

Travailler depuis le terminal avec Aiven m'a aidé à mieux retenir la syntaxe SQL, surtout DELIMITER pour créer les procédures. Je pourrai réutiliser ces notions dans mes projets, par exemple pour automatiser des calculs dans une base de données.